

# Aula 05 - Função sacar com regra mínima

---

## Contexto de negócio

Controle de risco: saque só pode passar quando atender validações de valor e saldo.

Como impedir que o programa retire mais dinheiro do que existe no saldo?

## Tensão operacional

Ainda não existe validação para saque.

## Objetivo técnico da entrega

- Regra condicional com retorno boolean.

## Recurso técnico desta aula

Método central desta etapa:

```
static boolean sacar(double p_valor, double p_saldo)
```

## Problema que justifica o novo artefato

No fluxo anterior, o saque ainda podia ficar implícito ou misturado com atualização direta. Precisamos explicitar uma decisão de aprovação antes de alterar saldo.

## Artefato explicado: regra mínima de saque com boolean

Artefato desta aula: `sacar(valor, saldo)` retornando `boolean`.

- `true`: operação autorizada;
- `false`: operação negada.

O algoritmo passa a depender de bifurcação explícita no chamador (`if/else`) para tratar sucesso e falha.

## Transição didática para a próxima aula

Depois de fixar o contrato de decisão, o próximo passo é aplicar a mesma lógica em múltiplas contas via estruturas indexadas.

## Valor prático entregue

- Reduz incidente de saque indevido.
- A base fica pronta para evolução controlada da próxima capacidade.

## Fluxo operacional explicado

- Preparar os dados de entrada do cenário da aula.
- Executar a regra principal e observar o impacto no estado.
- Confirmar saída de sucesso, erro e borda antes de concluir a etapa.

## Código em foco

```
// Classe temporaria de fluxo do programa.
// Neste curso, ela existe para iniciar a execução em Java.
public class AppGestaoContas {

    static double depositar(double p_saldo, double p_valor) {
        return p_saldo + p_valor;
    }

    static boolean sacar(double p_valor, double p_saldo) {
        return p_valor > 0 && p_valor <= p_saldo;
    }

    static void mostrarSaldo(double p_saldo) {
        System.out.println("Saldo atual: " + p_saldo);
    }

    public static void main(String[] args) {
        // Aula 05: regra simples de saque.
        double saldo = 1000.00;
        saldo = depositar(saldo, 200.00);

        if (sacar(150.00, saldo)) {
            saldo = saldo - 150.00;
            System.out.println("Saque realizado.");
        } else {
            System.out.println("Saque negado.");
        }

        mostrarSaldo(saldo);
    }
}
```

## Leitura técnica orientada

Percorra o código com estes checkpoints de revisão:

- Regra de decisão: mapear condições que aprovam ou negam operações.
- Saída observável: conferir mensagens que comprovam sucesso, erro e bloqueio de regra.

## Decisões de projeto das funções

1. **sacar** com ou sem **else** As duas abordagens podem ser corretas. Sem **else**, o fluxo tende a ficar mais direto com retorno padrão ao final; com **else**, alguns times acham a leitura mais explícita. O importante é manter consistência no projeto.
2. Regras em **depositar** Mesmo em uma aula simples, **depositar** pode validar:
  - valor positivo;
  - teto diário de depósito;
  - conta ativa.
3. Estabilidade de assinatura Quando a assinatura está bem definida (**depositar(saldo, valor)**), novas regras podem ser adicionadas dentro do método sem alterar quem chama.

## Discussão de contrato de retorno

Quando o método retorna **boolean**, o contrato precisa ser tratado como regra de negócio:

- **true** significa operação aprovada e estado atualizado;
- **false** significa operação negada e estado preservado.

Risco de lógica silenciosa:

- ignorar o **false** no chamador cria fluxo aparentemente correto, mas operacionalmente incompleto.

Responsabilidade de quem chama:

- sempre tratar os dois ramos (**true** e **false**) com mensagens e ações coerentes;
- registrar motivo de negativa quando possível (conta inativa, valor inválido, limite excedido);
- evitar seguir o fluxo como se a operação tivesse ocorrido.

Variações para debate técnico:

1. Em quais cenários **boolean** basta?
2. Quando o time precisa de retorno com motivo da falha?
3. Qual custo de não tratar o **false** em produção?

## Paradigma em foco: imperativo, procedural e algoritmo

Leitura de paradigma nesta etapa:

- Imperativo: o programa descreve passo a passo o que a máquina deve executar.
- Procedural: organizamos esse passo a passo em funções reutilizáveis.
- Algoritmo: sequência finita de entrada, processamento, decisão e saída para resolver um problema real.

Por que passar valores nas funções no procedural:

- A função não “conhece” a conta por contexto implícito.
- Tudo que a regra precisa deve chegar por parâmetro.
- O contrato da função fica explícito, testável e auditável.

Risco comum quando isso não fica claro:

- chamar função com parâmetro errado, em ordem errada ou incompleto;

- gerar resultado válido na sintaxe, mas incorreto no negócio.

Ponte para a próxima virada de paradigma:

- No procedural, o estado circula entre funções.
- Na POO, estado e comportamento passam a coexistir no objeto.

## Perguntas de decisão

1. Quando `sacar` retorna `true`?
2. Por que usar boolean neste ponto?
3. Onde o saldo é de fato alterado?
4. Em que situação você preferiria `else` explícito em `sacar`?
5. Quais regras de `depositar` cabem sem mudar assinatura?

## Variações de cenário

1. Testar saque com valor 0 e observar o retorno.
2. Testar saque com valor negativo e comparar com o caso anterior.

## Exercícios progressivos

1. Reescrever o `main` com dois saques seguidos, exibindo saldo após cada tentativa.
2. Alterar a regra para permitir saque até 10% acima do saldo e explicar o impacto no resultado.

## Desafio de equipe

Alterar `sacar` para retornar o novo saldo quando permitido e o saldo atual quando negado.

Objetivo didático: comparar abordagem com retorno boolean vs retorno de estado.

## CrITÉRIOS de aceite dos exercícios

- Regra principal continua válida após alterações.
- Cenário de erro foi testado e tratado sem quebrar execução.
- Código permanece legível e coerente com o nível da aula.

## Checklist de progressividade técnica

- Pré-requisito confirmado: leitura e execução do código-base da aula anterior.
- Novidade técnica identificada: explicar em uma frase o recurso novo desta aula.
- Complexidade controlada: apontar qual decisão de projeto evita acoplamento desnecessário.
- Evidência prática: executar ao menos um caso de sucesso e um caso de erro no Tryit.

## Fechamento executivo

Agora temos validação básica de saque. Na próxima, vamos escalar para várias contas.

## Revisão rápida (5 minutos)

1. Regra central: qual condição decide aprovação ou bloqueio na aula?

2. Risco real: qual erro operacional essa implementacao reduz?
3. Evolucao: qual pequena extensao agregaria mais valor sem aumentar acoplamento?

Mini-missao de fechamento (5 min):

- Execute um cenário de borda no Tryit e justifique o resultado em 3 linhas.

## Execução no W3Schools Tryit

1. Abrir o compilador online: [https://www.w3schools.com/java/tryjava.asp?filename=demo\\_compiler](https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler)
2. Colar todo o conteudo de AppGestaoContas.java no editor.
3. Clicar em Run, registrar saída base e depois testar variações e exercicios.
4. Manter uma aba separada para cada exercicio e comparar resultados.

## Referências W3Schools da aula

- [https://www.w3schools.com/java/java\\_booleans.asp](https://www.w3schools.com/java/java_booleans.asp)
- [https://www.w3schools.com/java/java\\_operators.asp](https://www.w3schools.com/java/java_operators.asp)
- [https://www.w3schools.com/java/java\\_compiler.asp](https://www.w3schools.com/java/java_compiler.asp)

## Leituras complementares

- <https://docs.oracle.com/javase/tutorial/>
- <https://dev.java/learn/>